

41174 Quantum Algorithms Project Report

Asymptotically Efficient Quantum Karatsuba Multiplication

Sibishan Ravindran

May 17, 2026

1 Computational Task

In the 3rd grade, we all learned how to multiply two integers. This “grade school” multiplication algorithm requires $\mathcal{O}(n^2)$ operations to multiply two n -bit integers. Karatsuba and Ofman [1] challenged this status quo by proposing a recursive divide-and-conquer algorithm, which runs in $\mathcal{O}(n^{\log_2 3})$ time. It came to be known as the Karatsuba algorithm. Algorithm 1 details the multiplication of two integers using Karatsuba’s method. We define the threshold for the base-case “grade school” multiplication as n_0 . Gidney [2] sets the threshold n_0 to 32 bits.

Algorithm 1 Classical Karatsuba multiplication

```
1: function KARATSUBAMULTIPLY( $u, v$ )
2:    $n \leftarrow \max(\text{BITLENGTH}(u), \text{BITLENGTH}(v))$ 
3:   if  $n \leq n_0$  then
4:     return  $u \cdot v$ 
5:   end if
6:    $h \leftarrow \lfloor n/2 \rfloor$ 
7:    $u_{\text{low}} \leftarrow u \bmod 2^h, \quad u_{\text{high}} \leftarrow \lfloor u/2^h \rfloor$ 
8:    $v_{\text{low}} \leftarrow v \bmod 2^h, \quad v_{\text{high}} \leftarrow \lfloor v/2^h \rfloor$ 
9:    $p_0 \leftarrow \text{KARATSUBAMULTIPLY}(u_{\text{low}}, v_{\text{low}})$ 
10:   $p_2 \leftarrow \text{KARATSUBAMULTIPLY}(u_{\text{high}}, v_{\text{high}})$ 
11:   $p_1 \leftarrow \text{KARATSUBAMULTIPLY}(u_{\text{low}} + u_{\text{high}}, v_{\text{low}} + v_{\text{high}}) - p_0 - p_2$ 
12:  return  $p_0 + p_1 \cdot 2^h + p_2 \cdot 2^{2h}$ 
13: end function
```

To build a quantum version of Algorithm 1, we cannot simply port the classical Karatsuba algorithm to be executed in a quantum computer [2]. To execute a quantum algorithm, we need to model a quantum circuit that performs the intended operations on the qubits. By nature, quantum circuits are reversible. Therefore, when we port a classical algorithm, we need to eliminate irreversible operations such as discarding variables or looping based on a condition [2]. Specifically, when we port Algorithm 1, we need to find a way to deal with the three recursive calls which produce discardable intermediate values. A naive approach to this is to uncompute the three recursive calls by again executing them in reverse order [2]. However, Gidney [2] points out that this will lead to $\mathcal{O}(n^{\log_2 6})$, which he rounds up to cubic, worse than

the “grade school” algorithm. Moreover, he points out the approach of Bennett [3], which involves storing all discardable intermediate results and uncomputing them at the end of the computation. Although this method at most requires twice as many operations as the classical Karatsuba algorithm, the need for intermediate calculation storage inflates space to $\mathcal{O}(n^{\log_2 3})$. The prior best-known quantum Karatsuba algorithm, due to Parent et al. [4] and building on Bennett’s pebble-game analysis [5], reduced the space complexity to $\mathcal{O}(n^{1.427})$ while preserving the time complexity of the classical Karatsuba algorithm. Gidney takes a different approach, which we will focus on, and achieves a space complexity of $\mathcal{O}(n)$ and $\mathcal{O}(n^{\log_2 3})$ gate count simultaneously, matching the asymptotic behaviour of the classical Karatsuba algorithm [2].

Multiplication is a foundational arithmetic primitive in any algorithm, whether classical or quantum. As discussed above, the Karatsuba algorithm provides a speedup over the “grade school” multiplication algorithm beyond the threshold n_0 . This algorithm can be used for the underlying modular multiplication primitive in Shor’s factoring algorithm [6]. However, the controlled modular multiplication in Shor’s factoring algorithm involves a classical integer and a quantum integer, not two quantum integers. Gidney [2] leaves the practical relevance of quantum Karatsuba multiplication open. Moreover, he acknowledges that the crossover at which Karatsuba multiplication outperforms “grade school” multiplication sits around 10,000 bits, well above the 2,048–8,192 bit range of modern RSA keys. Furthermore, he notes that his work does not account for the cost of routing qubits to enable their interaction. Gidney [2] proposes that his approach, *inline-mutation recursion*, is key to optimising the space complexity of quantum recursive algorithms. He considers it a basic tool for the quantum algorithm design toolbox.

2 Algorithm Sketch

Gidney [2] introduces *inline-mutation recursion* as the quantum counterpart of classical *tail-call optimisation*. Each recursive call writes directly into a slice of the caller’s output register instead of allocating its own temporary registers. Algorithm 2 details our own interpretation of Gidney’s inline Karatsuba multiplication method [2]. The procedure takes two input registers, U and V , an output register T , and a sign $s \in \{-1, +1\}$ that selects whether the product is added to or subtracted from T . Gidney [2] demonstrated that his inline Karatsuba multiplication algorithm can be executed in both classical and quantum computation settings. In this study, we focus on how his algorithm works in the quantum setting. For an n -bit integer encoded across m words of width w , the registers occupy:

$$|U| = m(w + \log_2 m) \text{ qubits}, \quad (1)$$

$$|V| = m(w + \log_2 m) \text{ qubits}, \quad (2)$$

$$|T| = 2m(2w + 3\log_2 m) \text{ qubits}. \quad (3)$$

Here, each word of U , V , and T is padded with extra qubits so that the carries produced by in-place additions stay within their word. Without this padding, every in-place addition would propagate a carry across the entire register, which would cost $\mathcal{O}(n)$ per operation, and make the

asymptotic improvement useless [2]. Moreover, Gidney [2] points out $w = \log_2 n$, $m = n/\log_2 n$ as the optimal word size and number of words, respectively, for asymptotic efficiency. Plugging his choices into Equations 1, 2, and 3 gives us $|U| = |V| \leq 2n$ and $|T| \leq 10n$. Therefore, the total qubit count is $\mathcal{O}(n)$ as promised.

Algorithm 2 Gidney’s inline Karatsuba multiplication

```

1: function ADDPRODUCTINTOPIECES( $U, V, T, s$ )
2:   input: word lists  $U, V$  of length  $m$ ; output word list  $T$  of length  $2m$ ; sign  $s \in \{-1, +1\}$ 
3:   if  $m = 1$  then
4:      $T[0] += U[0] \cdot V[0] \cdot s$ 
5:     return
6:   end if
7:    $h \leftarrow \lfloor m/2 \rfloor$ 
8:   for  $i = h$  to  $2m - 1$  do
9:      $T[i] += T[i - h]$ 
10:  end for
11:  ADDPRODUCTINTOPIECES( $U[0:h], V[0:h], T[0:2h], s$ )
12:  ADDPRODUCTINTOPIECES( $U[h:2h], V[h:2h], T[h:3h], -s$ )
13:  for  $i = 2m - 1$  down to  $h$  do
14:     $T[i] -= T[i - h]$ 
15:  end for
16:  for  $i = 0$  to  $h - 1$  do
17:     $U[i] += U[i + h]$ 
18:     $V[i] += V[i + h]$ 
19:  end for
20:  ADDPRODUCTINTOPIECES( $U[0:h], V[0:h], T[h:3h], s$ )
21:  for  $i = 0$  to  $h - 1$  do
22:     $U[i] -= U[i + h]$ 
23:     $V[i] -= V[i + h]$ 
24:  end for
25: end function

```

During inline mutations, Algorithm 2 refers to the given sub-registers of registers U , V , and T and performs operations on them. For example, the $T[h : 3h]$ “slice” represents the qubits encoding the h th through $(3h - 1)$ th words. When we pass a slice in a recursive call, the same physical qubits are passed as a reference to the recursive invocation, which writes its result into them through reversible gates. Therefore, the caller and the callee can share the same qubits during the call, and upon return, those qubits hold the updated state, without any ancilla qubits needed to store intermediate results.

Gidney [2] shows us that we can write the multiplication of two integers as:

$$t+ = (ax) \cdot (1 - 2^{wh}) - (by) \cdot 2^{wh} \cdot (1 - 2^{wh}) + (a + b)(x + y) \cdot 2^{wh}, \quad (4)$$

where $u = a + b \cdot 2^{wh}$ and $v = x + y \cdot 2^{wh}$. The factors 2^{wh} and $(1 - 2^{wh})$ in Equation 4 have to be avoided so that the recursive calls can mutate the output register inline [2]. But we need these factors to scale the results and get the correct answer. *I can show you a magic trick. It’s remarkable. Feast your eyes as I make them disappear*, thought Gidney. He converts the factors

of 2^{wh} into word-aligned shifts of the output slice, and the factors of $(1 - 2^{wh})$ into inverse-scaling self-additions of T before and after the recursive calls. These offsetting operations are performed in Algorithm 2 in between the recursion calls to compute the products ax and by . These offsetting operations can be clearly seen in Figure 1. This shows algorithm designers using inline-mutation recursion what to look out for when applying this technique.

Algorithm 2 is not a purely quantum program. It requires classical preprocessing of the two given inputs to compile a quantum circuit that performs reversible Karatsuba multiplication. Therefore, we divide this algorithm’s tasks into either the classical control flow required for compiling the quantum circuit or the quantum operations that perform the calculation. The major classical control flow is the recursion itself. Since the number of words m is known at compile time, the quantum circuit compiler can determine how many recursions are needed and when to enter the base case ($m = 1$) and perform “grade school” multiplication. In addition, all the loops defined in Algorithm 2 are part of circuit compilation. Each iteration emits one word-level adder into the circuit. Moreover, the padding schemes in Equations 1 to 3 are computed classically at the start to determine how many qubits are needed. Finally, the sign parameter s not only determines if the product produced in the base case should be added to or subtracted from the output register T , but also determines whether to produce the forward circuit to compute or the reverse circuit to uncompute.

If we were to look at the generated quantum circuit, it would only contain ripple-carry adders [7] implementing the in-place additions and subtractions between register slices, and Toffoli gates implementing the AND-based partial products at the base case [8]. On the other hand, Figure 1 gives us an overview of the steps performed by Gidney’s inline Karatsuba multiplication at one recursion level.

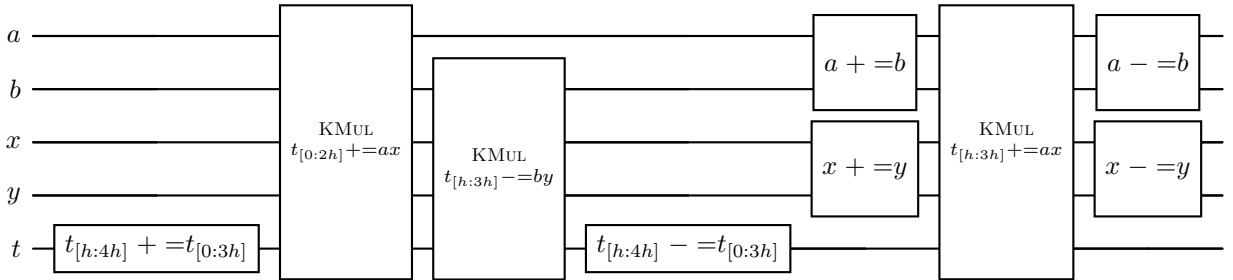


Figure 1: Our interpretation of one recursive level of Gidney’s inline Karatsuba multiplication, expanding the operation $t += uv$ into three half-sized recursive calls plus in-place scaling. Inputs are split as $u = a + b \cdot 2^{wh}$ and $v = x + y \cdot 2^{wh}$. Each KMUL box is a recursive call to this same block on operands of half the size, writing inline into the indicated slice of t . The recursion terminates at a base case where u and v fit in a single word, and a schoolbook multiplier is used instead.

However, Figure 1 does not give us the big picture on what’s needed to complete Gidney’s inline Karatsuba algorithm. This algorithm is designed to serve as a primitive in other algorithms for multiplying two quantum integers. So, T becomes a scratch register whose padding bits interleave with the true product bits. We know that qubits are a very limited resource in quantum computing. Therefore, to deliver the result to the caller’s output register and free T

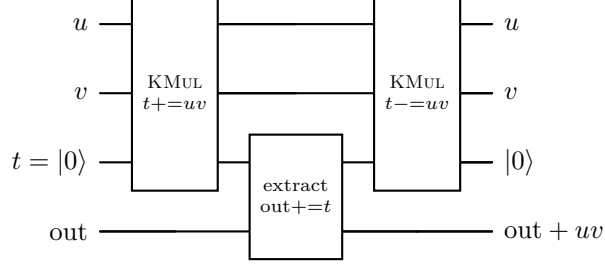


Figure 2: Our interpretation of the whole circuit for computing uv into scratch space t via the recursive step KMUL (Figure 1), extracts the result into the caller’s output register, then uncomputes t back to $|0\rangle$ with $\text{KMUL}^\dagger$.

for other computations, we need to extract the result and then uncompute T for reuse as shown in Figure 2.

3 Complexity Analysis

Gidney [2] claims his inline Karatsuba algorithm achieves $\mathcal{O}(n^{\log_2 3})$ gate complexity and $\mathcal{O}(n)$ space complexity simultaneously, matching the asymptotic behaviour of the classical Karatsuba algorithm. At the beginning of our algorithm sketch, we discussed how his *inline-mutation recursion* was able to stay in $\mathcal{O}(n)$ space without using any ancilla qubits.

Gidney [2] deduces that the total operation count is

$$\mathcal{O}\left(n^{\log_2 3} \frac{B(\log_2 n)}{(\log_2 n)^{\log_2 3}}\right)$$

where $B(x)$ is the cost of base-case multiplication on the x -bit operands. He points out that if we were to use “grade school” multiplication for the base case, the gate complexity would become $\mathcal{O}(n^{\log_2 3} \cdot \log_2^{0.42} n)$. This is not the asymptotic behaviour that he is aiming for. Fortunately, his choice of word size $w = \log_2 n$ means that the base case operates on $\mathcal{O}(\log_2 n)$ -bit operands. This allowed him to use a space-inefficient form of Karatsuba multiplication, such as [4] for the base case and achieve the gate complexity $\mathcal{O}(n^{\log_2 3})$. The additive space cost of this move is polylogarithmic in n and therefore asymptotically negligible.

The gate complexity is given for multiplying two n -bit integers using Karatsuba’s method. Our interpretation of the whole circuit in Figure 2 does not end with the forward pass; it continues on to use ripple-carry adders [7] to copy the result from the scratch register to the output register and execute the reverse pass to uncompute the scratch register. The extract step introduces $\mathcal{O}(n)$ additional gates and the reverse pass would require another $\mathcal{O}(n^{\log_2 3})$ gates. This brings us to a total operation count of

$$\mathcal{O}(n^{\log_2 3}) + \mathcal{O}(n) + \mathcal{O}(n^{\log_2 3}) = 2 \cdot \mathcal{O}(n^{\log_2 3}) + \mathcal{O}(n) = \mathcal{O}(n^{\log_2 3}).$$

When we are looking at the gate complexity asymptotically, the factor 2 and additional $\mathcal{O}(n)$ get absorbed into the big-O. Therefore, the whole circuit gate complexity is just $\mathcal{O}(n^{\log_2 3})$.

Although these additional gate costs are hidden in the big-O, they still affect factors such as circuit depth, error, decoherence, and so on. Gidney [2] further acknowledges that he did not account for additional SWAP gate costs arising from circuit routing required to execute on real quantum hardware.

Moreover, Gidney [2] proposes further optimisations for circuit depth, such as using log-depth adders, enabling parallel execution of base cases. However, he decided not to implement them in his simulation experiments due to a limitation in the Q# trace simulator: it cannot handle very large inputs. Therefore, he used “grade school” multiplication for the base case instead of a space-inefficient Karatsuba, since the latter offers no advantage at the input sizes he simulated. Thus, Figure 3 illustrates that his inline Karatsuba algorithm using “grade school” multiplication as the base case outperforms the “grade school” multiplication in terms of gate counts around the point where input size reaches 10,000 bits.

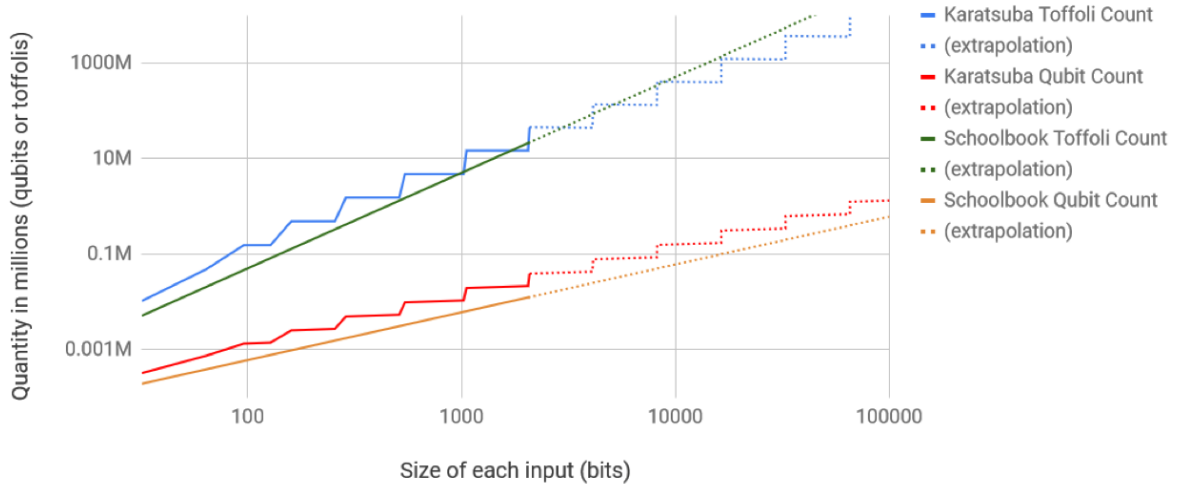


Figure 3: Toffoli and qubit counts for Gidney’s Karatsuba (with schoolbook base case) versus pure schoolbook multiplication, reproduced from [2].

4 Classical Competitor

It is trivial and much faster to compute multiplications in classical computers. Using quantum computers to perform multiplications as a standalone task is highly uneconomical. Gidney [2] introduced his inline Karatsuba multiplication as a proof-of-concept for his *inline-mutation recursion* technique. This study shows that this technique is the key to optimising the space complexity of any quantum recursive algorithm. Furthermore, he proposes his inline Karatsuba multiplication as a primitive to be used inside other quantum algorithms such as Shor’s factoring algorithm [6].

On the other hand, if we were to consider standalone Karatsuba multiplication for comparison between classical and quantum execution, even though both have the same complexity of $\mathcal{O}(n^{\log_2 3})$, classical computers using transistors can perform operations faster than quantum gate operations [9]. In addition, classical computation includes algorithms such as the

Schönhage–Strassen algorithm [10], and Harvey and van der Hoeven’s algorithm [11], which are far superior to the Karatsuba algorithm [1]. Although the Schönhage–Strassen algorithm has asymptotic complexity of $\mathcal{O}(n \log n \log \log n)$, the n has to be considerably larger to outperform the Karatsuba algorithm [10]. Similarly, the state-of-the-art Harvey and van der Hoeven’s algorithm has asymptotic complexity of $\mathcal{O}(n \log n)$, but the n has to be astronomically large to even outperform the Schönhage–Strassen algorithm, and is therefore referred to as a galactic algorithm [11].

It might seem fair to compare Gidney’s inline Karatsuba multiplication [2] against the classical Karatsuba algorithm [1] as a standalone computation and conclude that classical multiplication is faster. However, it is not that simple. This quantum Karatsuba multiplication is intended for use as a subroutine in larger quantum algorithms and, within the quantum setting, outperforms quantum “grade school” multiplication once the input exceeds roughly 10,000 bits. Even though Gidney’s algorithm does not use quantum properties such as superposition or interference, larger quantum algorithms like Shor’s factoring algorithm [6] do use them to demonstrate quantum advantage over classical computation. In this lens, the supremacy claim rests on those host algorithms, not on the multiplication itself.

5 Execution Environment

We deduce that Gidney’s inline Karatsuba algorithm [2] is designed for the fault-tolerant paradigm. The algorithm is purely reversible classical arithmetic executed on quantum registers. Its value lies in being a callable subroutine inside larger quantum algorithms such as Shor’s factoring algorithm [6], which themselves require fault tolerance.

Moreover, we know that this algorithm needs around 10,000-bit inputs to outperform quantum “grade-school” multiplication. Even if we wanted to perform this algorithm as a standalone task on currently available NISQ hardware, the padded qubit requirements shown in Equations 1-3 give approximately 140,000 logical qubits for multiplying two 10,000-bit integers. We also need to consider error correction. This could lead to a substantially larger physical qubit requirement, but the highest physical qubit count achieved to date is 6,100 [12]. Furthermore, given the high error rates and short coherence times of NISQ devices [13], it is a fruitless endeavour to make this algorithm work on NISQ computers.

Combined with our earlier observation that standalone multiplication is best left to classical computers, this rules out any practical standalone use of the algorithm. Our key takeaway from this study is that the *inline-mutation recursion* technique can be used to optimise space usage in quantum recursive algorithms, and Gidney’s inline Karatsuba algorithm is merely a proof of concept; Gidney leaves the algorithm’s practical applicability open.

References

- [1] A. Karatsuba and Y. Ofman, “Multiplication of many-digital numbers by automatic computers,” *Doklady Akademii Nauk SSSR*, vol. 145, no. 2, pp. 293–294, 1962.

- [2] C. Gidney, *Asymptotically efficient quantum karatsuba multiplication*, 2019. arXiv: [1904.07356 \[quant-ph\]](#).
- [3] C. H. Bennett, “Logical reversibility of computation,” *IBM J. Res. Dev.*, vol. 17, no. 6, pp. 525–532, Nov. 1973. DOI: [10.1147/rd.176.0525](#).
- [4] A. Parent, M. Roetteler, and M. Mosca, *Improved reversible and quantum circuits for karatsuba-based integer multiplication*, 2017. arXiv: [1706.03419 \[quant-ph\]](#).
- [5] C. H. Bennett, “Time/space trade-offs for reversible computation,” *SIAM Journal on Computing*, vol. 18, no. 4, pp. 766–776, 1989. DOI: [10.1137/0218053](#).
- [6] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, ser. SFCs ’94, USA: IEEE Computer Society, 1994, pp. 124–134. DOI: [10.1109/SFCs.1994.365700](#).
- [7] S. A. Cuccaro, T. G. Draper, S. A. Kutin, and D. P. Moulton, *A new quantum ripple-carry addition circuit*, 2004. arXiv: [quant-ph/0410184 \[quant-ph\]](#).
- [8] C. Gidney, *quantum-karatsuba-2019: Code for asymptotically efficient quantum karatsuba multiplication*, <https://github.com/Strilanc/quantum-karatsuba-2019>.
- [9] S. Choi, W. S. Moses, and N. Thompson, “The quantum tortoise and the classical hare: When will quantum computers outpace classical ones and when will they be left behind?” *Proceedings of the IEEE*, vol. 113, no. 2, pp. 113–124, 2025. DOI: [10.1109/JPROC.2025.3574102](#).
- [10] A. Schönhage and V. Strassen, “Schnelle multiplikation großer zahlen,” *Computing*, vol. 7, pp. 281–292, 1971. DOI: [10.1007/BF02242355](#).
- [11] D. Harvey and J. van der Hoeven, “Integer multiplication in time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021. DOI: [10.4007/annals.2021.193.2.4](#).
- [12] H. J. Manetsch, G. Nomura, E. Bataille, et al., “A tweezer array with 6,100 highly coherent atomic qubits,” *Nature*, vol. 647, pp. 60–67, 2025. DOI: [10.1038/s41586-025-09641-4](#).
- [13] J. Preskill, “Quantum Computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, Aug. 2018. DOI: [10.22331/q-2018-08-06-79](#).